Level 6 · about 30 h · Python 3.8+

Project: A tiny note-sorter that accepts --title
Examples run on CPython 3.12.3. **You will be able to:**

- Join path pieces with pathlib instead of gluing strings
- Find a number inside text with a regular expression
- Give a small program a command-line option with argparse
- Keep a sample from touching the real filesystem

Paths and Patterns

Paths are objects

`pathlib.Path` joins pieces with `/`. `as_posix` shows the forward-slash form, which is what the docs use in examples.

```
from pathlib import Path
path = Path("notes") / "week" / "a.txt"
print(path.as_posix())
print(path.suffix)
```

```
Output (CPython 3.12.3):
notes/week/a.txt
.txt
```

Patterns find a piece of text

`re.search` returns a match object, or `None` when nothing matched. This example looks for digits.

```
import re
match = re.search(r"\d+", "room 12b")
print(match.group(0) if match else "none")
print(re.sub(r"\s+", "-", "a  b   c"))
```

```
Output (CPython 3.12.3):
12
a-b-c
```

argparse reads options

`parse_args([])` reads an empty argument list, so the build does not depend on how the script was started. The default is what a user would get if they passed no flags.

```
import argparse
parser = argparse.ArgumentParser(prog="notes")
parser.add_argument("--title", default="inbox")
args = parser.parse_args([])
print(args.title)
```

```
Output (CPython 3.12.3):
inbox
```

Check yourself

1. What does `Path("notes") / "a.txt"` produce?
2. What does `re.search` return when the pattern is missing?
3. Why does this page call `parse_args([])`?

Sources

- Python docs: pathlib
- Python docs: re
- Python docs: argparse

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.